the specified entry point function returns, the thread exits. As you'll see, if you have a `std::thread` object for a thread, you can wait for it to finish; but first you have to start it, so let's look at launching threads.

2.1.1. Launching a thread

As you saw in chapter 1, threads are started by constructing a `std::thread` object that specifies the task to run on that thread. In the simplest case, that task is just a plain, ordinary `void`-returning function that takes no parameters. This function runs on its own thread until it returns, and then the thread stops. At the other extreme, the task could be a function object that takes additional parameters and performs a series of independent operations that are specified through some kind of messaging system while it's running, and the thread stops only when it's signaled to do so, again via some kind of messaging system. It doesn't matter what the thread is going to do or where it's launched from, but starting a thread using the C++ Thread Library always boils down to constructing a `std::thread` object:

```
void do_some_work();
std::thread my_thread(do_some_work);
```

This is just about as simple as it gets. Of course, you have to make sure that the `<thread>` header is included so the compiler can see the definition of the `std::thread` class. As with much of the C++ Standard Library, `std::thread` works with any *callable* type, so you can pass an instance of a class with a function call operator to the `std::thread` constructor instead:

```
class background_task
{
public:
    void operator()() const
     {
        do_something();
        do_something_else();
     }
};
background_task f;
std::thread my_thread(f);
```